

Focus IA

Guia para exponer el backend

Documento de referencia con las funciones, decisiones y endpoints mas importantes del backend.

Pensado para defender en una exposicion: que hace cada cosa, por que esta asi, y como responder preguntas.

Stack tecnologico

FastAPI + SQLAlchemy async + fastapi-users + JWT

MySQL (Railway) + Alembic + Docker

Pydantic v2 para validacion en bordes

1. Arquitectura general

El backend es una API REST con FastAPI que sirve a tres clientes: el frontend Angular (panel del usuario), la extension Chrome (que monitorea distractores en tiempo real) y los tests psicometricos (TDAH e IQ).

Componentes clave

- app/main.py: configura el engine async, session factory, UserManager, JWT y CORS.
- app/models/: SQLAlchemy. User extiende SQLAlchemyBaseUserTable[int] (fastapi-users).
- app/schemas/: Pydantic v2. Validacion en los bordes (HTTP <-> dominio).
- app/routes/: routers por feature (preferences, sessions, tdah, iq, profile).
- app/services/profile.py: helpers compartidos (normalize_level, get_or_create_profile).

Por que async

La extension Chrome hace polling cada pocos segundos para saber si la sesion sigue activa, ademas de POSTear cada deteccion. Con SQLAlchemy async + aiomysql el servidor no bloquea hilos esperando la BD; un solo worker uvicorn maneja muchas conexiones concurrentes.

Como hablar de auth

fastapi-users genera /auth/register, /auth/jwt/login, /users/me. El JWT se firma con JWT_SECRET (env var). Los routers propios protegen con la dependencia current_active_user.

2. Patrones que se repiten (los que mas preguntan)

2.1 Mutar User.profile_data (columna JSON)

SQLAlchemy NO detecta mutaciones in-place sobre columnas JSON. Si haces `user.profile_data['x'] = 1` directamente, el ORM no marca el campo como dirty y el cambio no se guarda.

Patron correcto:

```
data = dict(user.profile_data or {}) # 1. copia
data['mode_alias'] = 'alerta'        # 2. muta la copia
user.profile_data = data             # 3. asignacion (trigger ORM)
flag_modified(user, 'profile_data')  # 4. seguro extra
session.add(user)
await session.commit()
```

Frase para defender: 'JSON en SQLAlchemy es opaco al ORM, hay que reasignar el dict y avisar con `flag_modified`'.

2.2 Aliases vs niveles canonicos

El frontend muestra modos amigables (tranquilo / alerta / absoluta). La BD y la extension manejan niveles canonicos (bajo / intermedio / alto). Si no se mapea, el backend recibe 'alerta' y no lo reconoce -> todas las sesiones quedaban en 'intermedio' por default.

Solucion en `services/profile.py`:

```
ALIAS_TO_LEVEL = {
    'tranquilo': 'bajo',
    'alerta':    'intermedio',
    'absoluta':  'alto',
}

def normalize_level(value):
    if not value: return None
    v = value.strip().lower()
    if v in VALID_LEVELS: return v
    return ALIAS_TO_LEVEL.get(v)
```

2.3 Validacion en los bordes con Pydantic

En vez de validar dentro de la logica, se hace antes con `Field(ge=, le=)` y `Literal[...]`. Si llega algo invalido, FastAPI responde 422 antes de tocar BD.

```
class IqAnswerIn(BaseModel):
    questionId: int = Field(..., ge=1, le=30)
    selectedIndex: int = Field(..., ge=0, le=3)
    timeSpentMs: int = Field(..., ge=0)
```

3. Endpoints clave (con que decir de cada uno)

3.1 POST /preferences

Guarda el modo y duracion preferidos del usuario.

- Recibe el alias del frontend (tranquilo/alerta/absoluta).
- Lo normaliza a canonico (bajo/intermedio/alto).
- Guarda alias en profile_data['mode_alias'] (para que el GET devuelva lo que el frontend espera).
- Actualiza PerfilEstudiante.nivel_restriccion (fuente de verdad para la extension).
- Si hay sesion activa, sincroniza Sesion.nivel_restriccion_sesion.

3.2 GET /sessions/active

Devuelve la sesion activa con el nivel VIVO (preferencia actual del perfil), no el snapshot con el que arranco la sesion. Asi, si el usuario cambia su modo a la mitad, la extension lo detecta sin reiniciar.

```
perfil = await get_or_create_profile(session, user.id)
nivel_vivo = (
    perfil.nivel_restriccion
    or sesion.nivel_restriccion_sesion
    or 'intermedio'
)
```

Importante: el snapshot Sesion.nivel_restriccion_sesion NO se borra. Solo se sincroniza desde POST /preferences. Asi los registros historicos de deteccion conservan el nivel real del momento.

3.3 POST /tdah/resultados

Guarda el resultado del cuestionario TDAH (18 preguntas) en profile_data['tdah']. No se creo tabla porque solo interesa el ultimo resultado (YAGNI). Validacion estricta con Literal ['Muy bajo','Bajo','Moderado','Alto','Muy alto'].

3.4 POST /iq/resultados - la mas interesante de explicar

Hace tres cosas en UNA sola transaccion: guarda el resultado, ajusta preferencias automaticamente y sincroniza la sesion activa.

Anti-trampa (el punto fuerte):

El frontend manda cada respuesta con un flag isCorrect, pero el backend lo IGNORA y recalcula el score contra una tabla autoritativa CORRECT_INDEX. Si el cliente manipula isCorrect, no afecta el resultado.

```
score = 0
for a in payload.answers:
    if CORRECT_INDEX.get(a.questionId) == a.selectedIndex:
        score += 1
```

Auto-ajuste de preferencias:

< 40 % -> nivel Bajo	modo absoluta	25 min
40-70 % -> nivel Promedio	modo alerta	35 min
> 70 % -> nivel Alto	modo tranquilo	50 min

Logica: puntaje bajo necesita mas restriccion para concentrarse; puntaje alto ya autorregula bien, modo mas permisivo.

4. Preguntas que pueden hacerte (y como responder)

P: Por que guardas datos en JSON y no en columnas?

Para datos que no quiero indexar ni consultar por ellos (mode_alias, duration, frases motivacionales, resultado de tests). Son del usuario, los lee solo el; no necesito JOINS ni WHEREs sobre esos campos. Crear tablas para todo seria sobreingenieria. Cuando algo necesite historial o filtros, ahi si migra a tabla.

P: Por que el backend recalcula el score del IQ?

Anti-trampa. El frontend manda isCorrect en cada respuesta, pero el cliente puede modificarlo. El backend tiene la tabla CORRECT_INDEX como fuente de verdad y siempre recalcula. Es defensa en profundidad.

P: Por que /sessions/active devuelve el nivel del perfil y no el de la sesion?

Para que los cambios de preferencia sean inmediatos sin tener que reiniciar la sesion. La extension hace polling y al siguiente tick ya tiene el nivel nuevo. El snapshot de la sesion sigue ahi para los registros historicos de deteccion.

P: Que pasa si dos requests modifican profile_data al mismo tiempo?

Cada request abre su propia sesion async. La ultima en commit gana (last write wins). Para los casos de uso actuales (un usuario, operaciones poco frecuentes) es aceptable. Si se vuelve critico se puede usar SELECT ... FOR UPDATE o version columns.

P: Por que aliasas en vez de mostrar bajo/intermedio/alto?

UX. 'Modo tranquilo' comunica mejor que 'restriccion baja'. La separacion permite cambiar copys del frontend sin tocar logica de negocio ni la extension. El mapeo vive en un solo lugar (profile.py).

P: Por que async y no sync?

La extension Chrome hace polling y POSTea detecciones constantemente. Con sync, cada peticion bloquearia un thread esperando MySQL. Con async + aiomysql, un solo worker uvicorn maneja muchas conexiones concurrentes sin escalar verticalmente.

P: Por que fastapi-users en vez de tu propio auth?

Da gratis: registro, login con JWT, password hashing con bcrypt, users/me, dependencias de proteccion de rutas. Lo unico que sobrescribi es _update del UserManager para soportar el campo username (que vive en profile_data, no es columna).

5. Mini-checklist para la exposicion

Que mencionar SI o SI

- Stack: FastAPI async + SQLAlchemy async + fastapi-users + JWT + MySQL.
- Patron de mutacion de columnas JSON (copia + reasignacion + flag_modified).
- Mapeo de alias (era un bug: todas las sesiones quedaban en 'intermedio').
- Anti-trampa en el test de IQ (recalculo en backend con CORRECT_INDEX).
- Sesion con nivel vivo: preferencia se actualiza sin reiniciar sesion.
- Validacion en bordes con Pydantic (Field, Literal) -> 422 antes de tocar BD.

Frases listas para usar

- 'El backend es la fuente de verdad: aunque el frontend mande isCorrect, yo recalculo. Defensa en profundidad.'
- 'Alias en frontend, niveles canonicos en BD. El mapeo vive en un solo lugar para no acoplar UI con logica de dominio.'
- 'Para columnas JSON en SQLAlchemy hay que reasignar el dict y avisar con flag_modified, sino el ORM no detecta el cambio.'
- 'Cuestionario TDAH guarda solo el ultimo resultado en JSON. Si en el futuro hace falta historial, se migra a tabla. YAGNI.'

Si te bloqueas

Vuelve siempre al diagrama: el backend recibe peticiones de frontend Angular y de la extension Chrome, valida en bordes con Pydantic, normaliza datos, opera en transacciones y guarda en MySQL. Todo lo demas son optimizaciones.

Suerte mañana. Tu lo entiendes mejor de lo que crees.